

MYSQL Task

1. Retrieve all customers who have placed an order in the last 30 days.

```
SELECT DISTINCT c.*  
FROM customers c  
JOIN orders o ON c.id = o.customer_id  
WHERE o.order_date >= DATE_SUB(NOW(), INTERVAL 30 DAY);
```

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the SQL query: `SELECT DISTINCT c.* FROM customers c JOIN orders o ON c.id = o.customer_id WHERE o.order_date >= DATE_SUB(NOW(), INTERVAL 30 DAY);`. The 'Result Grid' shows the results of the query, which are the first three rows of the 'customers' table. The 'Navigator' on the left shows the database structure, including the 'ecommerce' database and its tables: 'customers', 'orders', and 'products'. The 'Object Info' panel at the bottom left shows the structure of the 'customers' table.

id	name	email	address
1	Alice Smith	alice@example.com	123 Apple St, NY
2	Bob Jones	bob@example.com	456 Banana Ave, LA
4	Diana Prince	diana@example.com	101 Date Ct, FL

Table: customers
Columns:
id int AI PK
name varchar(50)
email varchar(50)
address varchar(200)

2. Get the total amount of all orders placed by each customer.

**SELECT c.name, COALESCE(SUM(o.total_amount), 0) AS total_spent FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id GROUP BY c.id, c.name;**

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the 'ecommerce' database selected. The main editor shows a SQL query in 'Query 1'.

```
1 • select * from customers;  
2 • select * from products;  
3 • select * from orders;  
4  
5 • SELECT c.name, COALESCE(SUM(o.total_amount), 0) AS total_spent  
6 FROM customers c  
7 LEFT JOIN orders o ON c.id = o.customer_id  
8 GROUP BY c.id, c.name;
```

The 'Result Grid' at the bottom shows the output of the query:

name	total_spent
Alice Smith	225.00
Bob Jones	15.00
Charlie Brown	40.00
Diana Prince	25.00
Evan Wright	100.00

The bottom-left pane shows the 'Table: customers' structure:

Columns:

- id: int AI PK
- name: varchar(50)
- email: varchar(50)
- address: varchar(200)

3. Update the price of Product C to 45.00.

UPDATE products

SET price = 45.00

WHERE id =3; //name = 'Product C';

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL script:

```
1 select * from customers;
2 select * from products;
3 select * from orders;
4
5 UPDATE products
6 SET price = 45.00
7 WHERE id =3; //name = 'Product C';
```

The 'Output' tab shows the results of the queries:

#	Time	Action	Message	Duration / Fetch
24	19:00:52	SELECT c.name, COALESCE(SUM(o.total_amount), 0) AS total_spent FROM customers c	5 row(s) returned	0.000 sec / 0.000 sec
25	19:02:32	UPDATE products SET price = 45.00 WHERE name = 'Product C'	Error Code: 1175. You are using safe update mode and you tried to update a table without a ...	0.047 sec
26	19:03:12	select * from products LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
27	19:04:26	UPDATE products SET price = 45.00 WHERE id =3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.094 sec

4. Add a new column discount to the products table.

Before added new column:

The screenshot shows the MySQL Workbench interface. The 'Query' tab is active, displaying the following SQL script:

```
1 select * from customers;
2 select * from products;
3 select * from orders;
4
5 ALTER TABLE products
6 ADD COLUMN discount DECIMAL(5, 2) DEFAULT 0.00;
```

The 'Result Grid' shows the data in the 'products' table:

id	name	price	description
1	Product A	25.00	High-quality widget
2	Product B	15.00	Standard gadget
3	Product C	45.00	Premium tool
4	Product D	100.00	Advanced device
5	Product E	200.00	Luxury equipment

The 'Output' tab shows the results of the queries:

#	Time	Action	Message	Duration / Fetch
25	19:02:32	UPDATE products SET price = 45.00 WHERE name = 'Product C'	Error Code: 1175. You are using safe update mode and you tried to update a table without a ...	0.047 sec
26	19:03:12	select * from products LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
27	19:04:26	UPDATE products SET price = 45.00 WHERE id =3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.094 sec
28	19:06:17	select * from products LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

After added NEW Column:

ALTER TABLE products

ADD COLUMN discount DECIMAL(5, 2) DEFAULT 0.00;

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

b17

ecommerce

Tables

customers

orders

products

Views

Stored Procedures

Functions

fsd_b17

sys

Query 1

```
1 select * from customers;
2 select * from products;
3 select * from orders;
4
5 ALTER TABLE products
6 ADD COLUMN discount DECIMAL(5, 2) DEFAULT 0.00;
```

Result Grid

id	name	price	description	discount
1	Product A	25.00	High-quality widget	0.00
2	Product B	15.00	Standard gadget	0.00
3	Product C	45.00	Premium tool	0.00
4	Product D	100.00	Advanced device	0.00
5	Product E	200.00	Luxury equipment	0.00

Administration Schemas

Information

Table: customers

Columns:

id	name	email	address
int AI PK	varchar(50)	varchar(50)	varchar(200)

Object Info Session

Output

Action Output

#	Time	Action	Message	Duration / Fetch
27	19:04:26	UPDATE products SET price = 45.00 WHERE id =3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.094 sec
28	19:06:17	select * from products LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
29	19:07:15	ALTER TABLE products ADD COLUMN discount DECIMAL(5, 2) DEFAULT 0.00	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.422 sec
30	19:07:21	select * from products LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

5. Retrieve the top 3 products with the highest price.

```
SELECT *  
FROM products  
ORDER BY price DESC  
LIMIT 3;
```

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

b17

ecommerce

Tables

customers

orders

products

Views

Stored Procedures

Functions

fsd_b17

sys

Query 1

Limit to 1000 rows

```
1 select * from customers;  
2 select * from products;  
3 select * from orders;  
4  
5 SELECT *  
6 FROM products  
7 ORDER BY price DESC  
8 LIMIT 3;
```

Result Grid

	id	name	price	description	discount
▶	5	Product E	200.00	Luxury equipment	0.00
▶	4	Product D	100.00	Advanced device	0.00
▶	3	Product C	45.00	Premium tool	0.00
*		NULL	NULL	NULL	NULL

Administration Schemas

Information

Table: customers

Columns:

id int AI PK

name varchar(50)

email varchar(50)

address varchar(200)

Object Info Session

products 13

Output

Action Output

#	Time	Action	Message
✓	28 19:06:17	select * from products LIMIT 0, 1000	5 row(s) returned
✓	29 19:07:15	ALTER TABLE products ADD COLUMN discount DECIMAL(5, 2) DEFAULT 0.00	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
✓	30 19:07:21	select * from products LIMIT 0, 1000	5 row(s) returned
✓	31 19:12:30	SELECT * FROM products ORDER BY price DESC LIMIT 3	3 row(s) returned

6. Get the names of customers who have ordered Product A.

**SELECT DISTINCT c.name FROM customers c JOIN orders o ON c.id = o.customer_id
JOIN products p ON o.product_id = p.id WHERE p.name = 'Product A';**

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including tables like customers, orders, and products. The main editor window contains the following SQL query:

```
1 • select * from customers;
2 • select * from products;
3 • select * from orders;
4
5 • SELECT DISTINCT c.name
6   FROM customers c
7  JOIN orders o ON c.id = o.customer_id
8  JOIN products p ON o.product_id = p.id
9  WHERE p.name = 'Product A';
```

Below the query editor, the 'Result Grid' shows the output of the query:

name
Alice Smith
Diana Prince

The bottom section of the interface shows the 'Information' tab, which displays the table structure for 'customers':

Columns:	
id	int AI PK
name	varchar(50)
email	varchar(50)
address	varchar(200)

The 'Output' tab at the bottom shows a log of database actions, including the execution of the query and the number of rows returned.

#	Time	Action	Message
29	19:07:15	ALTER TABLE products ADD COLUMN discount DECIMAL(5, 2) DEFAULT 0.00	0 row(s) affected Records: 0 Dup
30	19:07:21	select * from products LIMIT 0, 1000	5 row(s) returned
31	19:12:30	SELECT * FROM products ORDER BY price DESC LIMIT 3	3 row(s) returned
32	19:14:04	SELECT DISTINCT c.name FROM customers c JOIN orders o ON c.id = o.customer_id JOI...	2 row(s) returned

7. Join the orders and customers tables to retrieve the customer's name and order date for each order.

SELECT c.name AS customer_name, o.order_date FROM orders o JOIN customers c ON o.customer_id = c.id;

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the 'ecommerce' database selected, showing tables 'customers', 'orders', and 'products'. The main editor window shows a SQL query in 'Query 1':

```
1 • select * from customers;
2 • select * from products;
3 • select * from orders;
4
5 • SELECT c.name AS customer_name, o.order_date
6 FROM orders o
7 JOIN customers c ON o.customer_id = c.id;
```

Below the query editor, the 'Result Grid' shows the output of the query:

customer_name	order_date
Alice Smith	2026-10-02 18:51:57
Alice Smith	2026-10-05 18:51:57
Bob Jones	2026-09-22 18:51:57
Charlie Brown	2026-08-23 18:51:57
Diana Prince	2026-09-27 18:51:57
Evan Wright	2026-08-08 18:51:57

The bottom section of the interface shows the 'Table: customers' structure:

Table: customers

Columns:

- id: int AI PK
- name: varchar(50)
- email: varchar(50)
- address: varchar(200)

The 'Output' pane at the bottom shows the execution log:

#	Time	Action	Message
31	19:12:30	SELECT * FROM products ORDER BY price DESC LIMIT 3	3 row(s) returned
32	19:14:04	SELECT DISTINCT c.name FROM customers c JOIN orders o ON c.id = o.customer_id JOI...	2 row(s) returned
33	19:15:11	SELECT c.name AS customer_name, o.order_date FROM orders o JOIN customers c ON o....	6 row(s) returned

8. Retrieve the orders with a total amount greater than 150.00.

```
SELECT *  
FROM orders  
WHERE total_amount > 150.00;
```

The screenshot shows the MySQL Workbench interface. The 'Query 1' tab is active, displaying the SQL query: `SELECT * FROM orders WHERE total_amount > 150.00;`. The 'Result Grid' shows the results of the query, which is currently empty. The 'Navigator' on the left shows the database structure, including the 'ecommerce' database and its tables: 'customers', 'orders', and 'products'. The 'Table: customers' information is visible at the bottom left, showing columns: 'id' (int AI PK), 'name' (varchar(50)), 'email' (varchar(50)), and 'address' (varchar(200)). The 'Output' tab at the bottom right shows the execution log, indicating that the query was executed successfully and returned 1 row(s).

id	customer_id	product_id	order_date	total_amount
4	1	5	2026-10-05 18:51:57	200.00
NULL	NULL	NULL	NULL	NULL

9. Normalize the database by creating a separate table for order items and updating the orders table to reference the order_items table.

- **CREATE TABLE order_items (id INT AUTO_INCREMENT PRIMARY KEY, order_id INT, product_id INT, quantity INT DEFAULT 1, price DECIMAL(10, 2), FOREIGN KEY (order_id) REFERENCES orders(id), FOREIGN KEY (product_id) REFERENCES products(id));**
- **INSERT INTO order_items (order_id, product_id, quantity, price) SELECT id, product_id, 1, total_amount FROM orders;**
- **ALTER TABLE orders DROP FOREIGN KEY fk_product; ALTER TABLE orders DROP COLUMN product_id;**

10. Retrieve the average total of all orders.

SELECT AVG(total_amount) AS average_order_total FROM orders;

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a tree view with 'b17' expanded, containing 'ecommerce' (Tables: customers, orders, products; Views; Stored Procedures; Functions) and 'fsd_b17'. The main editor shows a query with three lines: 'select * from customers;', 'select * from products;', and 'select * from orders;'. Below the query, the 'Result Grid' shows a single column 'average_order_total' with a value of '67.500000'. At the bottom, the 'Table: customers' structure is shown with columns: id (int AI PK), name (varchar(50)), email (varchar(50)), and address (varchar(200)). The 'Output' pane at the bottom right shows a log of actions, including 'SELECT c.name AS customer_name, o.order_date FROM orders o JOIN customers c ON o....' (6 row(s) returned), 'SELECT * FROM orders WHERE total_amount > 150.00 LIMIT 0, 1000' (1 row(s) returned), and 'SELECT AVG(total_amount) AS average_order_total FROM orders LIMIT 0, 1000' (1 row(s) returned).

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

b17

ecommerce

Tables

customers

orders

products

Views

Stored Procedures

Functions

fsd_b17

sys

Query 1 x

Limit to 1000 rows

```
1 • select * from customers;
2 • select * from products;
3 • select * from orders;
4
5 • SELECT AVG(total_amount) AS average_order_total
6   FROM orders;
```

Result Grid

average_order_total
67.500000

Administration Schemas

Information

Table: customers

Columns:

Column	Type	Attributes
id	int	AI PK
name	varchar(50)	
email	varchar(50)	
address	varchar(200)	

Object Info Session

Result 17 x

Output

Action Output

#	Time	Action	Message
33	19:15:11	SELECT c.name AS customer_name, o.order_date FROM orders o JOIN customers c ON o....	6 row(s) returned
34	19:16:14	SELECT * FROM orders WHERE total_amount > 150.00 LIMIT 0, 1000	1 row(s) returned
35	19:22:14	SELECT AVG(total_amount) AS average_order_total FROM orders LIMIT 0, 1000	1 row(s) returned